



Zadatak Mapa

Ana ima N parova prirodnih brojeva s vrijednostima između 1 i 10^9 . Dodatno, ne postoje dva para s istim prvim brojem. Dakle, Anine parove možemo zamišljati kao članove *mape* (npr. `map<int,int>` u C++-u). Prve brojeve u parovima zato nazivamo ključevima, a druge vrijednostima.

Ana želi poslati sadržaj svoje mape Borni, no komunikacijski kanal koji koristi ima ograničen broj bitova! Ana će zato morati smisliti postupak kojim će svoju mapu pretvoriti u niz bitova određene duljine koje će zatim poslati Borna preko komunikacijskog kanala.

Borna će primiti poslani niz bitova te iz njega pokušati rekonstruirati Aninu mapu. Iako možda neće znati sadržaj cijele mape, on mora biti u stanju ostvariti funkcionalnost mape. Preciznije, za svaku moguću vrijednost x koja **pripada skupu ključeva**, Borna mora moći vratiti odgovarajuću vrijednost `mapa[x]` tog ključa u mapi. Da bi bio siguran da je uspješno rekonstruirao mapu, Borna će se pitati za neku vrijednost ključa Q puta.

Pomozite Ani i Borna te napišite program koji danu mapu pretvara u niz bitova te koji iz danog niza bitova može rekonstruirati polaznu mapu.

Ulazni podaci

U prvom je retku prirodan broj T ($T = 1$ ili $T = 2$). Ako je $T = 1$, program mora kodirati danu mapu u niz bitova. Ako je $T = 2$, program mora iz danog niza bitova rekonstruirati polaznu mapu.

Ako je $T = 1$, u sljedećem se retku nalazi prirodni broj N ($1 \leq N \leq 100$), broj parova u Aninoj mapi. Zatim, u i -tom od sljedećih N redaka su prirodni brojevi x_i i y_i ($1 \leq x_i, y_i \leq 10^9$) koji predstavljaju redom ključ i vrijednost jednog para u mapi. Vrijedi $x_i \neq x_j$ za $i \neq j$.

Ako je $T = 2$, u sljedećem se retku nalaze tri prirodna broja N, Q i K ($1 \leq N, Q \leq 100, 1 \leq K \leq 6000$), redom broj parova u mapi, broj upita koji si Borna postavlja te broj primljenih bitova. U sljedećem se retku nalazi niz znakova 0 ili 1 duljine K koji predstavlja primljeni niz bitova. U i -tom od sljedećih Q redaka je prirodni broj x_i . Garantirano je da x_i pripada skupu mogućih ključeva polazne mape.

Izlazni podaci

Ako je $T = 1$, u prvom retku ispišite prirodni broj K ($1 \leq K \leq 6000$), duljinu kodiranog niza bitova. U sljedećem retku ispišite niz znakova 0 ili 1, redom vrijednosti poslanih bitova.

Ako je $T = 2$, ispišite Q redaka. U i -ti redak ispišite odgovor na i -ti upit.

Bodovanje

Vaše će rješenje biti testirano u dva koraka. Prvo će biti pozvano sa službenim ulaznim podacima u kojima je $T = 1$. Ako je izlaz vašeg rješenja validan niz od K bitova, u drugom koraku vaše će rješenje biti ponovno pokrenuto s nizom bitova ispisanim u prvom koraku. Ako su odgovori na upite koje vaš program ispiše u drugom koraku u skladu s mapom iz službenih ulaznih podataka, smatra se da je vaš program uspješno rekonstruirao mapu.

Ukoliko vaš program nije uspješno rekonstruirao mapu, bodovat će se s 0 bodova. Inače, broj ostvarenih bodova ovisi o K po uzoru na sljedeću tablicu:

Raspon	Broj bodova
$K > 6000$	0
$3000 \leq K \leq 6000$	$100 - \frac{K-3000}{30}$
$K \leq 3000$	100



Vrijeme izvršavanja vašeg rješenja je zbroj vremena izvršavanja oba koraka evaluacije.

Primjer interakcije

ulaz

1
3
2 10
3 3
5 7

izlaz

7
1100111

ulaz

2
3 2 7
1100111
5
3

izlaz

7
3



Ancient Machine

Anna and Bruno are archaeologists. They excavate ruins of IOI Kingdom. In the ruin A, Anna discovered the layout of an old machine. In the ruin B, Bruno discovered the actual machine.

This machine consists of N devices. The devices are attached to an electric wire in a row. There are three types of devices called X, Y, Z. From the left, the devices are numbered from 0 through $N - 1$. The type of the device i ($0 \leq i \leq N - 1$) is S_i . In other words, S_i is either X, Y, or Z.

Since the machine is too big, Bruno decided to remove the devices, one by one, from the machine. However, since the devices interact with each other by an electric wire, he must be very careful about the order of removal.

Concerning a way to remove a device from the machine, we define the following.

- Assume that the devices x, y, z ($0 \leq x < y < z \leq N - 1$) are not yet removed, and $S_x = X$, $S_y = Y$, $S_z = Z$. Moreover, assume that, for every $x < j < y$, the device j was already removed, and, for every $y < k < z$, the device k was already removed. If all of these conditions are satisfied and we remove the device y from the machine, it is called a **good removal**.
- Any other way to remove a device from the machine is not a good removal.

Bruno has to remove all of the N devices from the machine so that the number of good removals is the maximum possible. However, since the three types of devices look similar, he cannot distinguish the types of the devices.

Since Anna has the layout of the machine, she knows the type of each device attached to the machine. Thus, she will use a transmitter to help Bruno. Using the transmitter, she can send a sequence of characters. Each character she can send is either 0 or 1.

Write a program which implements Anna's strategy and Bruno's strategy so that the number of good removals is the maximum possible. In this task, if the number of characters sent by Anna to Bruno is smaller, your will get higher score.

Implementation Details

You need to submit two files.

The first file is `Anna.cpp`. It should implement Anna's strategy. It should implement the following function. The program should include `Anna.h` using the preprocessing directive `#include`.

- `void Anna(int N, std::vector<char> S)`

For each test case, this function is called exactly once in the beginning.



- The parameter N is the number of devices N .
- The parameter S is an array of length N . This means $S[i]$ is the type S_i of the device i ($0 \leq i \leq N - 1$). Here the character $S[i]$ is either 'X', 'Y', or 'Z'.

Your program can call the following function.

★ `void Send(int a)`

Using this function, Anna sends a character, 0 or 1, to Bruno.

- The parameter a is an information sent to Bruno. It should be 0 or 1. If this condition is not satisfied, your program is judged as **Wrong Answer [1]**.
- The function `Send` should not be called more than 200 000 times. If it is called more than 200 000 times, your program is judged as **Wrong Answer [2]**.

The second file is `Bruno.cpp`. It should implement Bruno's strategy. It should implement the following function. The program should include `Bruno.h` using the preprocessing directive `#include`.

• `void Bruno(int N, int L, std::vector<int> A)`

After the function `Anna` is called, this function is called exactly once.

- The parameter N is the number of devices N .
- The parameter L is the number L of characters, 0 or 1, sent by Anna.
- The parameter A is an array of length L . It means Anna sent the sequence $A[0], A[1], \dots, A[L-1]$ of characters to Bruno, in this order. Each character of the sequence is either 0 or 1.

Your program can call the following function.

★ `void Remove(int d)`

Using this function, your program answers how to remove the devices.

- The parameter d is the index of a device. It means Bruno removes the device d .
- The inequalities $0 \leq d \leq N - 1$ should be satisfied. If this condition is not satisfied, your program is judged as **Wrong Answer [3]**.
- It is not allowed to call this function with same parameter d more than once. If this condition is not satisfied, your program is judged as **Wrong Answer [4]**.
- The function `Remove` should be called exactly N times. When the function `Bruno` terminates, if the number of calls to the function `Remove` is different from N , your program is judged as **Wrong Answer [5]**.
- After all the N devices are removed, the number of good removals should be the maximum possible. If this condition is not satisfied, your program is judged as **Wrong Answer [6]**.



Important Notices

- Your program can implement other functions for internal use, or use global variables. Submitted files will be compiled with the grader, and become a single executable file. All global variables and internal functions should be declared in an unnamed namespace to avoid conflict with other files. When it is graded, it will be executed as two processes of Anna and Bruno. The process of Anna and the process of Bruno cannot share global variables.
- Your program must not use the standard input and the standard output. Your program must not communicate with other files by any methods. However, your program may output debugging information to the standard error.

Compilation and Test Run

You can download an archive file from the contest webpage which contains the sample grader to test your program. The archive file also contains a sample source file of your program.

The sample grader is the file `grader.cpp`. In order to test your program, put `grader.cpp`, `Anna.cpp`, `Bruno.cpp`, `Anna.h`, `Bruno.h` in the same directory, and run the following command to compile your programs.

```
g++ -std=gnu++17 -O2 -fsigned-char -o grader grader.cpp Anna.cpp Bruno.cpp
```

When the compilation succeeds, the executable file `grader` is generated.

Note that the actual grader is different from the sample grader. The sample grader will be executed as a single process, which will read input data from the standard input and write the results to the standard output.

Input for the Sample Grader

The sample grader reads the following data from the standard input.

$$N$$
$$S_0 \ S_1 \ \cdots \ S_{N-1}$$

Here S_i and S_{i+1} ($0 \leq i \leq N-2$) are separated by a space.



Output of the Sample Grader

When the program terminates successfully, the sample grader writes the following information to the standard output (quotes for clarity).

- If your program is judged as any one of Wrong Answer [1], [2], [3], [4], or [5], it writes its type as “Wrong Answer [1]”.
- Otherwise, it writes the number L of function calls to Send and the number D of good removals as “Accepted: L D”. Note that the behavior of the sample grader is different from the actual grader. The sample grader does not check whether your program is judged as Wrong Answer [6] or not.

If your program is judged as several types of Wrong Answer [1], [2], [3], [4], or [5], the sample grader reports only one of them.

Constraints

- $3 \leq N \leq 100\,000$.
- S_i is one of X, Y, or Z ($0 \leq i \leq N - 1$).

Subtasks

1. (5 points) $N \leq 18$.
2. (95 points) No additional constraints. In this Subtask, your score is calculated by the following way.
 - Let L be the maximum of the number of function calls to Send for every test case of this Subtask.
 - Then your score is calculated as follows.
 - If $160\,000 < L \leq 200\,000$, your score is $25 + \left\lfloor 10 \times \frac{200000 - L}{40000} \right\rfloor$ points.
 - If $100\,000 < L \leq 160\,000$, your score is $35 + \left\lfloor 30 \times \frac{160000 - L}{60000} \right\rfloor$ points.
 - If $70\,000 < L \leq 100\,000$, your score is $65 + \left\lfloor 30 \times \left(\frac{100000 - L}{30000} \right)^2 \right\rfloor$ points.
 - If $L \leq 70\,000$, your score is 95 points.

Here $\lfloor x \rfloor$ is the largest integer not exceeding x .



Sample Communication

Here is a sample input for the sample grader and corresponding function calls.

Sample Input 1	Sample Function Calls	
	Call	Call
4 X Y X Z	Anna(4, {X, Y, X, Z})	
		Send(0)
		Send(1)
	Bruno(4, 2, {0, 1})	
		Remove(2)
		Remove(1)
		Remove(0)
		Remove(3)

In these sample function calls, the 4 devices will be removed by the following way.

1. In the beginning, the 4 devices are X Y X Z.
2. Bruno removes the device 2. The devices become X Y - Z. Here - means the device in this position was already removed.
3. Bruno removes the device 1. The devices become X - - Z. Since $(x, y, z) = (0, 1, 3)$ satisfies the condition, it is a good removal.
4. Bruno removes the device 0. The devices become - - - Z.
5. Finally, Bruno removes the device 3. The devices become - - - -.

The number of good removals is 1.

In this sample input, the number of good removals cannot be greater than 1.



Two Transportations

There are N cities in JOI country, numbered from 0 to $N - 1$. There are A railway lines, numbered from 0 to $A - 1$. The railway line i ($0 \leq i \leq A - 1$) connects the city U_i and the city V_i bidirectionally, and its fare is C_i . Different railway lines connect different pairs of cities. There are B bus lines, numbered from 0 to $B - 1$. The bus line j ($0 \leq j \leq B - 1$) connects the city S_j and the city T_j bidirectionally, and its fare is D_j . Different bus lines connect different pairs of cities, but a railway line and a bus line might connect the same pair of cities. It is possible to travel between any pair of cities by taking railways and/or buses.

Azer wants to know the minimum total fare needed to travel from the city 0 to each city. As Azer knows only about railway lines, he cooperates with Baijan, who knows only about bus lines.

They communicate with each other by sending and receiving characters 0 or 1. The total number of the sent characters should be less than or equal to 58 000.

Write programs which, Azer's program given the information of railway lines, and Baijan's given the information of bus lines, communicating with each other, help Azer find the minimum total fare needed to travel from the city 0 to each city.

Implementation Details

You need to submit two files.

The name of the first file is `Azer.cpp`. It represents the behavior of Azer and should implement the following functions. The file should include `Azer.h`.

- `void InitA(int N, int A, std::vector<int> U, std::vector<int> V, std::vector<int> C)`

This function is executed exactly once at the beginning.

- Parameter N is N , the number of cities.
- Parameter A is A , the number of railway lines.
- Parameters U, V are arrays of length A . $U[i]$ and $V[i]$ are U_i and V_i , the cities connected by the railway line i ($0 \leq i \leq A - 1$).
- Parameter C is an array of length A . $C[i]$ is C_i , the fare of the railway line i ($0 \leq i \leq A - 1$).

- `void ReceiveA(bool x)`

This function is executed every time a character is sent from Baijan.

- The parameter x represents the character sent from Baijan: `true` for character 1, and `false` for character 0.



- `std::vector<int> Answer()`

This function is executed exactly once when all the sent characters have been received. This function must return an array Z containing the minimum total fare needed to travel from the city 0 to each city.

- The return value Z must be an array of length N . If its length is not N , your program is judged as **Wrong Answer [1]**. $Z[k]$ ($0 \leq k \leq N - 1$) must be the minimum total fare needed to travel from the city 0 to the city k . In particular, note that $Z[0] = 0$ must be satisfied.

Your program can call the following function in this file.

- ★ `void SendA(bool y)`

Use this function to send a character to Baijan.

- The parameter y represents the character to send to Baijan: `true` for character 1, and `false` for character 0.

The name of the second file is `Baijan.cpp`. It represents the behavior of Baijan and should implement the following functions. The file should include `Baijan.h`.

- `void InitB(int N, int B, std::vector<int> S, std::vector<int> T, std::vector<int> D)`

This function is executed exactly once at the beginning.

- Parameter N is N , the number of cities.
- Parameter B is B , the number of bus lines.
- Parameters S, T are arrays of length B . $S[j]$ and $T[j]$ are S_j and T_j , the cities connected by bus line j ($0 \leq j \leq B - 1$).
- Parameter D is an array of length B . $D[j]$ is D_j , the fare of bus line j ($0 \leq j \leq B - 1$).

- `void ReceiveB(bool y)`

This function is executed every time a character is sent from Azer.

- The parameter y represents the character sent from Azer: `true` for character 1, and `false` for character 0.

Your program can call the following function in this file.

- ★ `void SendB(bool x)`

Use this function to send a character to Azer.

- The parameter x represents the character to send to Azer: `true` for character 1, and `false` for character 0.



You can assume that the program is executed as follows. For each test case, two queues are prepared: Q_Y , where characters Azer sends are stored, and Q_X , where characters Baijan sends are stored. First, `InitA` and `InitB` are executed, and the sent characters are pushed to the corresponding queues.

- If either Q_X or Q_Y is not empty, one character is popped from it and corresponding `ReceiveA` or `ReceiveB` is executed. However, if both Q_X and Q_Y are not empty, it is not determined whether `ReceiveA` or `ReceiveB` is executed.
- When `SendA` is executed during executions of `ReceiveA`, the sent character is pushed to Q_Y .
- When `SendB` is executed during executions of `ReceiveB`, the sent character is pushed to Q_X .
- If both queues are empty, `Answer` is executed and the program ends.

The total number of characters Azer and Baijan send should be less than or equal to 58 000. If it is larger, your program is judged as **Wrong Answer [2]**.

Important Notices

- Your program can implement other functions for internal use, or use global variables. Submitted files will be compiled with the grader, and become a single executable file. All global variables and internal functions should be declared in an unnamed namespace to avoid confliction with other files. When it is graded, it will be executed as two processes of Azer and Baijan. The process of Azer and the process of Baijan cannot share global variables.
- Your program should not use the standard input and the standard output. Your program should not communicate with other files by any methods. However, your program may output debugging information to the standard error.

Compilation and Test Run

You can download an archive file from the contest webpage which contains the sample grader to test your program. The archive file also contains sample source files of your program.

The sample grader is the file `grader.cpp`. In order to test your program, put `grader.cpp`, `Azer.cpp`, `Baijan.cpp`, `Azer.h`, and `Baijan.h` in the same directory, and run the following command to compile your program.

```
g++ -std=gnu++14 -O2 -o grader grader.cpp Azer.cpp Baijan.cpp
```

If the compilation succeeds, the executable file `grader` is generated.



Note that the actual grader is different from the sample grader. The sample grader will be executed as a single process, which will read input data from the standard input and write the results to the standard output and the standard error.

Input for the Sample Grader

The sample grader reads the input data from the standard input in the following format.

$$\begin{array}{l} N \ A \ B \\ U_0 \ V_0 \ C_0 \\ \vdots \\ U_{A-1} \ V_{A-1} \ C_{A-1} \\ S_0 \ T_0 \ D_0 \\ \vdots \\ S_{B-1} \ T_{B-1} \ D_{B-1} \end{array}$$

Output of the Sample Grader

The sample grader writes the following information to the standard output and the standard error (quotes for clarity).

- If your program is judged as Wrong Answer [1] or Wrong Answer [2], it writes its type as “Wrong Answer [1]” to the standard error. It writes nothing to the standard output.
- Otherwise, it writes the total number of the sent characters L as “Accepted: L” to the standard error. It also writes your answer Z to the standard output as follows:

$$\begin{array}{l} Z[0] \\ \vdots \\ Z[N - 1] \end{array}$$

The sample grader does not check if the value of Z is correct.

If your program is judged as several types of Wrong Answer, the sample grader reports only one of them.



Constraints

- $1 \leq N \leq 2\,000$.
- $0 \leq A \leq 500\,000$.
- $0 \leq B \leq 500\,000$.
- $0 \leq U_i \leq N - 1$ ($0 \leq i \leq A - 1$).
- $0 \leq V_i \leq N - 1$ ($0 \leq i \leq A - 1$).
- $U_i \neq V_i$ ($0 \leq i \leq A - 1$).
- $(U_{i_1}, V_{i_1}) \neq (U_{i_2}, V_{i_2})$ and $(U_{i_1}, V_{i_1}) \neq (V_{i_2}, U_{i_2})$ ($0 \leq i_1 < i_2 \leq A - 1$).
- $0 \leq S_j \leq N - 1$ ($0 \leq j \leq B - 1$).
- $0 \leq T_j \leq N - 1$ ($0 \leq j \leq B - 1$).
- $S_j \neq T_j$ ($0 \leq j \leq B - 1$).
- $(S_{j_1}, T_{j_1}) \neq (S_{j_2}, T_{j_2})$ and $(S_{j_1}, T_{j_1}) \neq (T_{j_2}, S_{j_2})$ ($0 \leq j_1 < j_2 \leq B - 1$).
- It is possible to travel between any pair of cities by taking railways and/or buses.
- $1 \leq C_i \leq 500$ ($0 \leq i \leq A - 1$).
- $1 \leq D_j \leq 500$ ($0 \leq j \leq B - 1$).

Subtasks

1. (6 points) $A = 0$.
2. (8 points) $B \leq 1\,000$.
3. (8 points) $A + B = N - 1$.
4. (38 points) $N \leq 900$.
5. (14 points) $N \leq 1\,100$.
6. (10 points) $N \leq 1\,400$.
7. (16 points) No additional constraints.



Sample Communication

Here is a sample input for the sample grader and corresponding function calls.

Sample Input 1	Sample Calls		
	Call	Call	Return
4 3 4 0 1 6 2 1 4 2 0 10 1 2 3 3 1 1 3 2 3 3 0 7	InitA(4, 3, {0,2,2}, {1,1,0}, {6,4,10})		
		SendA(true)	
		SendA(false)	
	InitB(4, 4, {1,3,3,3}, {2,1,2,0}, {3,1,3,7})		
	ReceiveB(true)		
		SendB(true)	
	ReceiveA(true)		
	ReceiveB(false)		
	Answer()		{0,6,9,7}

Magic Show

Alice and Bob are famous magicians. Catherine, a wealthy woman who showed interest in their amazing deeds, declared that she would give them huge wealth if they could perform the following magic trick. The contents of the magic are as follows:

- Step 1: Bob enters a room and completely cuts off from the outside. Bob can only communicate with Catherine. Then, Alice tells Catherine a number n between 2 and 5 000.
- Step 2: Catherine tells a number X to Alice, which is between 1 and 10^{18} .
- Step 3: Alice makes a tree with exactly n vertices, and gives it to Catherine.
- Step 4: Catherine deletes at most $\lfloor \frac{n-2}{2} \rfloor$ edges from the tree, and gives the remaining edges to Bob.
- Step 5: Bob carefully observes the graph, and tell the number which Catherine told to Alice.

However, Alice and Bob don't think they are smart enough to successfully perform this magic trick, so they are seeking your help. Please write a program which implements Alice's strategy and Bob's strategy so that they can beat Catherine's challenge.

Implementation Details

You need to submit **two files**:

The first file is `Alice.cpp`, which implements Alice's strategy. It should include `Alice.h` using the preprocessing directive `#include`. The function that needs to be implemented in the file is:

```
std::vector<std::pair<int, int>> Alice();
```

- For each test case, this function is called exactly once in the beginning.
- The function should return a vector of pairs, which represents the edges in the tree Alice constructed in Step 3 of the magic.
 - Note that the nodes of the tree should be numbered starting from 1.
 - You need to ensure that the returned tree is compliant, which means there should be exactly $n - 1$ edges and all nodes should be connected.

The function `Alice()` should call the following function **exactly once**:

```
long long setN(int n);
```

- Using this function, Alice chooses the parameter n which she gave to Catherine in Step 1 of the magic.
- The function then returns the value X , which Catherine gave to Alice in Step 2 of the magic.

The second file is `Bob.cpp`, which implements Bob's strategy. It should include `Bob.h` using the preprocessing directive `#include`. The function that needs to be implemented in the file is:

```
long long Bob(std::vector<std::pair<int, int>> V);
```

- For each test case, this function is called exactly once after the call of function `Alice()`.
- The parameter V is the list of edges of the graph Catherine gave to Bob in Step 4 of the magic.
- The edges are given **in sorted order**, which means:
 - For the two endpoints of each edge, the smaller numbered endpoint comes first;
 - All edges are sorted in ascending order based on the first endpoint being the first keyword and the second endpoint being the second keyword.
- The function should return a single integer, which represents the number X .

Sample Interaction

Call	Return Value
<code>Alice()</code>	
<code>setN(4)</code>	3
	$\{\{1, 2\}, \{2, 3\}, \{2, 4\}\}$
<code>Bob(\{\{1, 2\}, \{2, 4\}\})</code>	3

It represents the following scenario:

- Step 1: At first, Alice gives 4 to Catherine.
- Step 2: Catherine gives 3 to Alice.
- Step 3: Alice makes a tree with 4 nodes and edges $\{\{1, 2\}, \{2, 3\}, \{2, 4\}\}$, and tells it to Catherine.
- Step 4: Catherine cut the edge connecting nodes 2 and 3, and gives the remaining edges $\{\{1, 2\}, \{2, 4\}\}$ to Bob.
- Step 5: Bob tells the number 3. Because his answer is correct, they can successfully perform the magic show.

Constraints

- $1 \leq X \leq 10^{18}$.

Subtasks

1. (5 points): $X \leq 5\,000$.
2. (30 points): $X \leq 25\,000\,000$.
3. (65 points): No additional constraints.

Sample Grader

The sample grader reads the input in the following format:

- Line 1: T ($T \in \{1, 2\}$)

if $T = 1$, then the sample grader reads as follows:

- Line 2: X ($1 \leq X \leq 10^{18}$)

The sample grader prints your answer of function `Alice()` in the following format:

- Line 1: n
- Line $2 + i$ ($0 \leq i \leq n - 2$): $u[i] \ v[i]$, where there exists an edge connecting $u[i], v[i]$.

if $T = 2$, then The sample grader reads as follows:

- Line 2: $n \ m$ ($2 \leq n \leq 5000, n - 1 - \lfloor \frac{n-2}{2} \rfloor \leq m \leq n - 1$), where n is number of vertices, and m is number of remaining edges.
- Line $3 + i$ ($0 \leq i \leq m - 1$): $u[i] \ v[i]$, which means there exists an edge connecting $u[i], v[i]$.

The sample grader prints your answer of function `Bob()` in the following format:

- Line 1: X

Problem E. 网络恢复

Time Limit 4000 ms

Mem Limit 524288 kB

这是一道交互题。

出题人 02 喜欢网上冲浪。可是这天，他所在的小区的网络坏掉了，于是他喊来了你帮忙修一修。

小区的网络由 N 个网络结点和 M 条信道组成，可以被看作是一张 N 个点 M 条边的无向简单图（简单图满足任意两点之间至多存在一条直接相连的边，且没有自环）。点从 $1 \sim N$ 编号，边从 $1 \sim M$ 编号。目前，你只知道信道的总数是 M ，并且还掌握着每条信道的管理权限，然而你并不知道每条信道连接着哪两个结点。

为了恢复出网络结构，你可以使用一种土办法：重启大法！

当然重启也是需要智慧的。具体来说，你可以进行若干次操作，每次操作方式如下：

1. 给每个结点 i 标上一个自己定的权值 a_i ；
2. 选取一个信道的子集 S ，把不在 S 里的信道都关闭，只让 S 里的信道保持开启状态；
3. 此时，每个结点 i 会自动计算出与 i 通过开启状态的信道直接相邻的所有点 v 的 a_v 异或和，记为 b_i ；
4. 你通过管理员权限获取所有结点的 b_i 值，然后关闭的信道都重启，网络恢复至原状。

请你在不超过 50 次操作内，求出所有信道构成的集合。

注意，你只需要求出信道的集合。即，你只需要恢复出哪些结点之间有信道，不用恢复出每条信道对应的编号。

实现细节

你不需要，也不应该实现主函数，你只需要实现函数 `Solve(N, M)`，这里的 N 和 M 分别表示结点和信道的个数。你可以通过调用如下函数来和交互库进行交互：

1. Query(A, S)

- 这个函数可以暂时删去编号不在 S 中的信道，并将 i 号点的 a_i 设置成 $A[i - 1]$ ；
- 交互库会返回一个大小为 N 的 `vector` B ， $B[i - 1]$ 表示 i 号点的 b_i ；
- 你需要保证对 S 中的任意元素 x ，有 $1 \leq x \leq M$ 且互不相同，同时你需要保证 A 的大小恰好为 N ；
- A, S, B 均为 `vector`；
- A, B 中元素类型为 `unsigned long long`， S 中元素类型为 `int`。

2. Report(x, y)

- 这个函数可以向交互库汇报一条你发现的连接 x 和 y 的信道；

- 你需要保证 $1 \leq x, y \leq N$;
- x, y 均为 `int` ;
- 你必须恰好将所有信道报告一次，否则你的程序将会判定为错误。

评测时，交互库会恰好调用 `Solve` 一次。

本题保证所使用的图在交互开始之前已经完全确定，不会根据和你的程序的交互过程动态构造，因此题目中的交互操作都是确定性的，你不需要关心这些操作在交互库中的具体实现。

生成数据时，图是按一定方式从 N 个点， M 条边的简单无向图中随机选取的（见限制与约定一节）。

数据保证在调用次数限制下，交互库运行所需的时间不超过 2.5s；交互库使用的内存大小固定，且不超过 128MB。

实现方法

本题仅支持 C++ 语言。附加文件中已经提供了一个 `template_explore.cpp`，请将这个文件拷贝一份，重命名为 `explore.cpp`，然后在其基础上答题。

- 请确保你的程序开头有 `#include "explore.h"`。
- 你需要实现的函数 `Solve` 的接口信息如下：
 - `void Solve(int N, int M);`
- 你可以调用的交互函数的接口如下：
 - `vector<unsigned long long> Query(vector<unsigned long long> A, vector<int> S);`
 - `void Report(int x, int y);`

测试程序方式

试题目录下的 `grader.cpp` 是我们提供的交互库参考实现，最终测试时所用的交互库实现与该参考实现有所不同，因此选手的解法不应该依赖交互库实现。

1. 编译方法：

- 你需要在本题目录下使用如下命令编译得到可执行程序：
 - `g++ grader.cpp explore.cpp -o explore -O2 -lm`

2. 对于编译得到的可执行程序：

- 可执行文件将从标准输入读入以下格式的数据：
 - 第一行包含两个整数 N, M ，意义如题面描述。
 - 接下来 M 行，每行两个整数 x, y ，描述一条连接 x 号点与 y 号点的无向边。
- 读入完成之后，交互库将调用恰好一次函数 `Solve`，用输入的数据测试你的函数。你的函数正确返回后，交互库会判断你的计算是否正确，若正确则会输出 `Correct`，否则会输出相应的错误信息。

假设可执行文件读入的数据为：

```
1 | 3 2
2 | 1 2
3 | 2 3
```

数据第一行的两个整数分别表示点数和边条数，即 $N = 3, M = 2$ 。

`Query(S, A)` 调用次数不能超过 50 次。

下面是一个正确的交互过程：

选手程序	交互库	说明
	调用 <code>Solve(3, 2)</code>	开始测试
调用 <code>Query({1, 2, 4}, {1})</code>	返回 <code>{2, 1, 0}</code>	1 号节点权值为 $a[2] = 2$, 2号节点权值为 $a[1] = 1$, 3号节点权值为 0
调用 <code>Query({1, 2, 4}, {1, 2})</code>	返回 <code>{2, 5, 2}</code>	1 号节点权值为 $a[2] = 2$, 2号节点权值为 $a[1] \text{ xor } a[3] = 5$, 3号节点权值为 $a[2] = 2$
调用 <code>report(1, 2)</code>		发现了信道 (1, 2) 并记录
调用 <code>report(3, 2)</code>		发现了信道 (3, 2) 并记录
运行结束并返回	向屏幕打印 <code>Correct</code>	交互结束，结果正确

样例一

见附加文件下载

样例二

见附加文件下载

评分方式

最终评测只会收取 `explore.cpp`，修改选手目录下其他文件对评测无效。

本题首先会受到和传统题相同的限制。例如编译错误会导致整道题目得 0 分，运行时错误、超过时间限制、超过空间限制等会导致相应测试点得 0 分等。你只能访问自己定义的和交互库给出的变量及其对应

的内存空间，尝试访问其他空间将可能导致编译错误或运行错误。

在上述条件基础上，在一个测试点中，你得到满分，当且仅当：

1. 你的每次函数调用均合法，且调用 **Query**, **Report** 的次数分别不超过 50, M 次。
2. 由于 **Report** 的调用次数限制为 M ，你的每次调用都必须记录一条新的且存在的边；即每次调用 **report**(x , y) 时，应满足：有一条连接 x 号点和 y 号点的路径，且在这次调用之前从未调用过 **report**(x , y) 或 **report**(y , x)。
3. 你实现的函数 **Solve** 正常返回。
4. 在 **Solve** 函数返回时，你已经通过调用 **Report** 记录了全部 M 条信道。

限制与约定

测试点编号	N	M	特殊性质
1	$N = 20$	$M = 50$	无
2	$N = 60$	$M = 300$	
3	$N = 3000$	$M = 10000$	
4	$N = 6000$	$M = 30000$	
5	$N = 49998$	$M = 49997$	图为连通图
6	$N = 49999$	$M = 49999$	
7	$N = 50000$	$M = 50000$	无
8		$M = 300000$	
9	$N = 15000$		
10	$N = 10000$		

对于 5, 6 号测试点，保证按照如下方式随机生成：

- 均匀随机一条不在图中的非自环的边 (u, v) 。
- 如果加入 (u, v) 后，无论之后再怎么加边，都无法使得最终的图是个连通图，那么就不加入这条边；否则加入这条边。
- 重复操作直至加入恰好 M 条边为止。

对于其他测试数据，保证按照如下方式随机生成：

- 均匀随机一条不在图中的非自环的边 (u, v) 。
- 重复操作直至加入恰好 M 条边为止。

时间限制：4s

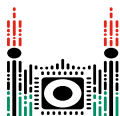
空间限制：512MB

提示

你的程序可以通过判断传入的 N 的个位来区分上述不同的数据类型。

下载

[附加文件下载](#)



Longest Trip

The IOI 2023 organizers are in big trouble! They forgot to plan the trip to Ópusztaszer for the upcoming day. But maybe it is not yet too late ...

There are N landmarks at Ópusztaszer indexed from 0 to $N - 1$. Some pairs of these landmarks are connected by *bidirectional roads*. Each pair of landmarks is connected by at most one road. The organizers *don't know* which landmarks are connected by roads.

We say that the **density** of the road network at Ópusztaszer is **at least** δ if every 3 distinct landmarks have at least δ roads among them. In other words, for each triplet of landmarks (u, v, w) such that $0 \leq u < v < w < N$, among the pairs of landmarks (u, v) , (v, w) and (u, w) at least δ pairs are connected by a road.

The organizers *know* a positive integer D such that the density of the road network is at least D . Note that the value of D cannot be greater than 3.

The organizers can make **calls** to the phone dispatcher at Ópusztaszer to gather information about the road connections between certain landmarks. In each call, two nonempty arrays of landmarks $[A[0], \dots, A[P - 1]]$ and $[B[0], \dots, B[R - 1]]$ must be specified. The landmarks must be pairwise distinct, that is,

- $A[i] \neq A[j]$ for each i and j such that $0 \leq i < j < P$;
- $B[i] \neq B[j]$ for each i and j such that $0 \leq i < j < R$;
- $A[i] \neq B[j]$ for each i and j such that $0 \leq i < P$ and $0 \leq j < R$.

For each call, the dispatcher reports whether there is a road connecting a landmark from A and a landmark from B . More precisely, the dispatcher iterates over all pairs i and j such that $0 \leq i < P$ and $0 \leq j < R$. If, for any of them, the landmarks $A[i]$ and $B[j]$ are connected by a road, the dispatcher returns `true`. Otherwise, the dispatcher returns `false`.

A **trip** of length l is a sequence of *distinct* landmarks $t[0], t[1], \dots, t[l - 1]$, where for each i between 0 and $l - 2$, inclusive, landmark $t[i]$ and landmark $t[i + 1]$ are connected by a road. A trip of length l is called a **longest trip** if there does not exist any trip of length at least $l + 1$.

Your task is to help the organizers to find a longest trip at Ópusztaszer by making calls to the dispatcher.

Implementation Details

You should implement the following procedure:

```
int[] longest_trip(int N, int D)
```

- N : the number of landmarks at Ópusztaszer.
- D : the guaranteed minimum density of the road network.
- This procedure should return an array $t = [t[0], t[1], \dots, t[l - 1]]$, representing a longest trip.
- This procedure may be called **multiple times** in each test case.

The above procedure can make calls to the following procedure:

```
bool are_connected(int[] A, int[] B)
```

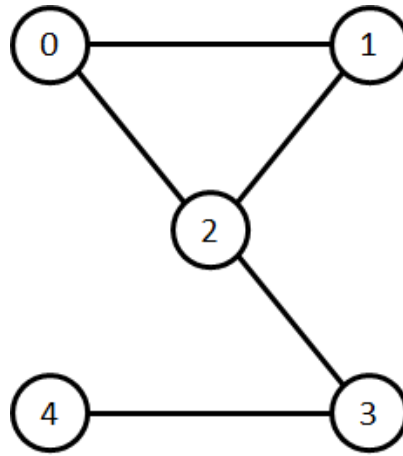
- A : a nonempty array of distinct landmarks.
- B : a nonempty array of distinct landmarks.
- A and B should be disjoint.
- This procedure returns `true` if there is a landmark from A and a landmark from B connected by a road. Otherwise, it returns `false`.
- This procedure can be called at most 32 640 times in each invocation of `longest_trip`, and at most 150 000 times in total.
- The total length of arrays A and B passed to this procedure over all of its invocations cannot exceed 1 500 000.

The grader is **not adaptive**. Each submission is graded on the same set of test cases. That is, the values of N and D , as well as the pairs of landmarks connected by roads, are fixed for each call of `longest_trip` within each test case.

Examples

Example 1

Consider a scenario in which $N = 5$, $D = 1$, and the road connections are as shown in the following figure:



The procedure `longest_trip` is called in the following way:

```
longest_trip(5, 1)
```

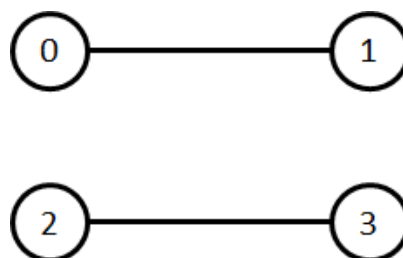
The procedure may make calls to `are_connected` as follows.

Call	Pairs connected by a road	Return value
<code>are_connected([0], [1, 2, 4, 3])</code>	(0,1) and (0,2)	true
<code>are_connected([2], [0])</code>	(2,0)	true
<code>are_connected([2], [3])</code>	(2,3)	true
<code>are_connected([1, 0], [4, 3])</code>	none	false

After the fourth call, it turns out that *none* of the pairs (1,4), (0,4), (1,3) and (0,3) is connected by a road. As the density of the network is at least $D = 1$, we see that from the triplet (0,3,4), the pair (3,4) must be connected by a road. Similarly to this, landmarks 0 and 1 must be connected.

At this point, it can be concluded that $t = [1, 0, 2, 3, 4]$ is a trip of length 5, and that there does not exist a trip of length greater than 5. Therefore, the procedure `longest_trip` may return `[1,0,2,3,4]`.

Consider another scenario in which $N = 4$, $D = 1$, and the roads between the landmarks are as shown in the following figure:



The procedure `longest_trip` is called in the following way:


```
longest_trip(4, 1)
```

In this scenario, the length of a longest trip is 2. Therefore, after a few calls to procedure `are_connected`, the procedure `longest_trip` may return one of $[0, 1]$, $[1, 0]$, $[2, 3]$ or $[3, 2]$.

Example 2

Subtask 0 contains an additional example test case with $N = 256$ landmarks. This test case is included in the attachment package that you can download from the contest system.

Constraints

- $3 \leq N \leq 256$
- The sum of N over all calls to `longest_trip` does not exceed 1 024 in each test case.
- $1 \leq D \leq 3$

Subtasks

1. (5 points) $D = 3$
2. (10 points) $D = 2$
3. (25 points) $D = 1$. Let l^* denote the length of a longest trip. Procedure `longest_trip` does not have to return a trip of length l^* . Instead, it should return a trip of length at least $\left\lceil \frac{l^*}{2} \right\rceil$.
4. (60 points) $D = 1$

In subtask 4 your score is determined based on the number of calls to procedure `are_connected` over a single invocation of `longest_trip`. Let q be the maximum number of calls among all invocations of `longest_trip` over every test case of the subtask. Your score for this subtask is calculated according to the following table:

Condition	Points
$2\,750 < q \leq 32\,640$	20
$550 < q \leq 2\,750$	30
$400 < q \leq 550$	45
$q \leq 400$	60

If, in any of the test cases, the calls to the procedure `are_connected` do not conform to the constraints described in Implementation Details, or the array returned by `longest_trip` is incorrect, the score of your solution for that subtask will be 0.

Sample Grader

Let C denote the number of scenarios, that is, the number of calls to `longest_trip`. The sample grader reads the input in the following format:

- line 1: C

The descriptions of C scenarios follow.

The sample grader reads the description of each scenario in the following format:

- line 1: $N\ D$
- line $1 + i$ ($1 \leq i < N$): $U_i[0]\ U_i[1]\ \dots\ U_i[i - 1]$

Here, each U_i ($1 \leq i < N$) is an array of size i , describing which pairs of landmarks are connected by a road. For each i and j such that $1 \leq i < N$ and $0 \leq j < i$:

- if landmarks j and i are connected by a road, then the value of $U_i[j]$ should be 1;
- if there is no road connecting landmarks j and i , then the value of $U_i[j]$ should be 0.

In each scenario, before calling `longest_trip`, the sample grader checks whether the density of the road network is at least D . If this condition is not met, it prints the message `Insufficient Density` and terminates.

If the sample grader detects a protocol violation, the output of the sample grader is `Protocol Violation: <MSG>`, where `<MSG>` is one of the following error messages:

- `invalid array`: in a call to `are_connected`, at least one of arrays A and B
 - is empty, or
 - contains an element that is not an integer between 0 and $N - 1$, inclusive, or
 - contains the same element at least twice.
- `non-disjoint arrays`: in a call to `are_connected`, arrays A and B are not disjoint.
- `too many calls`: the number of calls made to `are_connected` exceeds 32 640 over the current invocation of `longest_trip`, or exceeds 150 000 in total.
- `too many elements`: the total number of landmarks passed to `are_connected` over all calls exceeds 1 500 000.

Otherwise, let the elements of the array returned by `longest_trip` in a scenario be $t[0], t[1], \dots, t[l - 1]$ for some nonnegative l . The sample grader prints three lines for this scenario in the following format:

- line 1: l
- line 2: $t[0]\ t[1]\ \dots\ t[l - 1]$
- line 3: the number of calls to `are_connected` over this scenario

Finally, the sample grader prints:

- line $1 + 3 \cdot C$: the maximum number of calls to `are_connected` over all calls to `longest_trip`



Ancient Machine 2

Bitaro and Bibako are archaeologists who excavated and investigated the ruin of JOI Kingdom. In the ruin, Bitaro found an old stone slate, and Bibako found an old machine.

From the results of the study, Bitaro found out that a string S of length N was written on the stone slate. Each character of the string is either '0' or '1'. However, he does not yet know each character of the string S .

On the other hand, from the results of the study, Bibako found out how to use the machine. To use it, we put the stone slate on the machine, input an integer m and two sequences of integers a, b , and make a query. Here the integer m and the sequences of integers a, b should satisfy the following conditions:

- The integer m is between 1 and 1 002, inclusive.
- Both of the lengths of the sequences a, b are equal to m .
- Every element of the sequences a, b is an integer between 0 and $m - 1$, inclusive.

If we put the stone slate on the machine, input an integer m and two sequences of integers a, b , and make a query, the machine will operate as follows and will show an integer.

1. The machine sets 0 in the **memory area** of the machine.
2. The machine performs the following N **operations**. The $(i + 1)$ -th ($0 \leq i \leq N - 1$) operation proceeds as follows.

Let x be the current integer set in the memory area of the machine. The machine reads the character S_i . Here, S_i is the i -th character of the string S , if we count the characters of the string S so that the first character is the 0-th character.

- If S_i is '0', the machine sets a_x in the memory area of the machine.
Here, a_x is the x -th element of the sequence a , if we count the elements of the sequence a so that the first element is the 0-th element.
- If S_i is '1', the machine sets b_x in the memory area of the machine.
Here, b_x is the x -th element of the sequence b , if we count the elements of the sequence b so that the first element is the 0-th element.

3. The machine shows the integer which is finally set in the memory area.

Using the machine, Bitaro wants to specify the string written on the stone slate. However, since the machine is very fragile, the number of queries cannot exceed 1 000. Moreover, the maximum of the integer m input to the machine for a query should be as small as possible.

Write a program which, using the machine, specifies the string written on the stone slate.



Implementation Details

You need to submit one file.

The file is `ancient2.cpp`. It should implement the following function. The program should include `ancient2.h` using the preprocessing directive `#include`.

- `std::string Solve(int N)`

This function is called only once for each test case. This function should return a string which is the same as the string S written on the stone slate.

- The parameter N is the length N of the string S written on the stone slate.
- This function should return a string of length N . If the length is different from N , your program is judged as **Wrong Answer [1]**.
- Each character of the return value is either '0' or '1'. If this condition is not satisfied, your program is judged as **Wrong Answer [2]**.
- The return value should be the same as the string S written on the stone slate. If this condition is not satisfied, your program is judged as **Wrong Answer [3]**.

Your program can call the following function.

- ★ `int Query(int m, std::vector<int> a, std::vector<int> b)`

Using this function, your program can make a query.

- ◊ The parameter m is the integer m input to the machine.
- ◊ The parameters a, b are the two sequences of integers a, b input to the machine.
- ◊ The return value is the integer shown by the machine when we put the stone slate on the machine, input the above integer and sequences, and make a query.
- ◊ The parameter m should be between 1 and 1 002, inclusive. If this condition is not satisfied, your program is judged as **Wrong Answer [4]**.
- ◊ Both of the lengths of the parameters a, b are equal to m . If this condition is not satisfied, your program is judged as **Wrong Answer [5]**.
- ◊ Every element of the parameters a, b is between 0 and $m - 1$, inclusive. If this condition is not satisfied, your program is judged as **Wrong Answer [6]**.
- ◊ The function `Query` should not be called more than 1 000 times. If it is called more than 1 000 times, your program is judged as **Wrong Answer [7]**.



Important Notices

- Your program can implement other functions for internal use, or use global variables.
- Your program must not use the standard input and the standard output. Your program must not communicate with other files by any methods. However, your program may output debugging information to the standard error.

Compilation and Test Run

You can download an archive file from the contest webpage which contains the sample grader to test your program. The archive file also contains a sample source file of your program.

The sample grader is the file `grader.cpp`. In order to test your program, put `grader.cpp`, `ancient2.cpp`, `ancient2.h` in the same directory, and run the following command to compile your programs. Instead, you may run `compile.sh` contained in the archive file.

```
g++ -std=gnu++17 -O2 -o grader grader.cpp ancient2.cpp
```

When the compilation succeeds, the executable file `grader` is generated.

Note that the actual grader is different from the sample grader. The sample grader will be executed as a single process, which will read input data from the standard input and write the results to the standard output.

Input for the Sample Grader

The sample grader reads the following data from the standard input.

```
N  
S
```

Output of the Sample Grader

The sample grader outputs the following information to the standard output (quotes for clarity).

- If your program is judged as correct, it reports the maximum of the parameter `m` of the function `Query` as “Accepted: 22”. However, if your program is judged as correct without calling the function `Query`, it outputs “Accepted: 0”.
- If your program is judged as any type of Wrong Answer, the sample grader writes its type as “Wrong



Answer [4]”.

If your program satisfies the conditions of several types of Wrong Answer, the sample grader reports only one of them.

Constraints

All the input data satisfy the following conditions.

- $N = 1\,000$.
- S is a string of length N .
- Each character of the string S is either ‘0’ or ‘1’.

Notices for Grading

For every test case, the actual grader is **not** adaptive. This means the grader has a fixed answer in the beginning.

Grading

If your program is judged as Wrong Answer for any of the test cases, your score for this task is 0.

If your program is judged as correct for all the test cases, your score is calculated by M , which is the maximum of the parameter m of the function `Query` for all the test cases. However, if your program is judged as correct without calling the function `Query`, your score is calculated by $M = 0$.

- If $103 \leq M \leq 1\,002$, your score is $10 + \left\lfloor \frac{(1002-M)^2}{9000} \right\rfloor$ points.
- If $0 \leq M \leq 102$, your score is 100 points.

Here $\lfloor x \rfloor$ is the largest integer not exceeding x .



Sample Communication

Here is a sample input for the sample grader and corresponding function calls.

Sample Input 1	Sample Function Calls			
	Function Call to Solve	Return Values	Function Call to Query	Return Values
3 110	Solve(3)			
			Query(4, [3, 3, 2, 2], [2, 2, 1, 0])	3
			Query(2, [0, 1], [1, 0])	0
			Query(1, [0], [0])	0
			Query(3, [1, 1, 1], [1, 1, 1])	1
		"110"		

Assume that the string S written on the stone slate is "110". If we put the stone slate on the machine, input $(m, a, b) = (4, [3, 3, 2, 2], [2, 2, 1, 0])$, and make a query, the machine will operate as follows.

1. The machine sets 0 in the memory area of the machine.
2. For the first operation, since S_0 is '1', the machine sets b_0 , i.e. 2, in the memory area of the machine.
3. For the second operation, since S_1 is '1', the machine sets b_2 , i.e. 1, in the memory area of the machine.
4. For the third operation, since S_2 is '0', the machine sets a_1 , i.e. 3, in the memory area of the machine.
5. Since the integer which is finally set in the memory area is 3, the machine shows 3.

Note that this sample input **does not satisfy** the constraint $N = 1\,000$.

Among the files which can be obtained from the contest webpage, `sample-02.txt` satisfies the constraint.



Road Service

Output Only Task

There are N cities in IOI kingdom, numbered from 1 to N . There are also $N - 1$ bidirectional roads, numbered from 1 to $N - 1$. The i -th road connects the A_i -th city and the B_i -th city. There exists a path between any pair of vertices.

The distance between two cities are defined as the smallest number of roads which connect the two cities. The total distance of IOI kingdom is defined as the sum of the distances between all pair of different cities.

The king of IOI kingdom plans to construct K additional roads in order to reduce the total distance and improve convenience.

You, as an assistant of the king, help the king by finding a good plan.

Task

Given the information of existing roads in IOI kingdom and the number of roads to construct, output a plan for constructing K roads. The less the total distance is, the more points you gain.

Input

There are 6 inputs for this task. Read the following data from each input.

- The first line of input contains three space separated integers N , K and W_0 . These mean that IOI kingdom has N cities and that the king plans to construct K roads. W_0 is a parameter for scoring.
- The i -th ($1 \leq i \leq N - 1$) line of the following $N - 1$ lines contains integers A_i and B_i , the cities which the i -th road connects.

Output

Write K lines on your submission file.

The j -th line of the output contains two integers X_j, Y_j ($1 \leq X_j \leq N$, $1 \leq Y_j \leq N$), representing the cities connected by a road to construct.

Submission

Submit the output for each of the inputs. The output is valid only if it follows the format on the Output section above.



Constraints

All input data satisfy the following conditions.

- $1 \leq N \leq 1\,000$.
- $1 \leq A_i < B_i \leq N$ ($1 \leq i \leq N - 1$).
- $(A_i, B_i) \neq (A_k, B_k)$ ($1 \leq i < k \leq N - 1$).
- There are a path between any pair of cities.

Scoring

For each inputs, your score is calculated as follows:

If your output does not follow the format, your score is zero point. Otherwise, let the total distance after constructing the roads according to your plan be W , and let the point for the input be P . Let us define

$$S = 1.0 - \frac{W}{W_0}$$

Here, the score you gain for the input is

$$\min(P, P \times 20^S)$$

The score for this task is the sum of the scores for each input, rounded to the nearest integer.

The values of N, K, W_0, P on each input are as follows:

Input Data	N	K	W_0	P
1	20	4	512	10
2	1000	100	2650000	18
3	1000	300	1755000	18
4	1000	100	2900000	18
5	1000	100	2690000	18
6	1000	300	1745000	18



Sample Input and Output

Sample Input 1	Sample Output 1
4 1 8 1 2 2 3 3 4	1 4

In addition to the existing roads, by constructing a road connecting the 1-st city and the 4-th city, the total distance becomes 8.

Let $P = 10$ for this input. Here, $S = 0$, thus the score is 10 points.

Sample Input 2	Sample Output 2
4 1 8 1 2 2 3 3 4	1 2

In this case, the total distance is 10 after constructing the road.

Let $P = 10$ for this input. Here, $S = -0.25$, thus the score is $4.728 \dots$ points.

Problem I. 百万富翁

Time Limit 6000 ms

Mem Limit 524288 kB

这是一道交互题

小 Y 的银行有 N 个客户，编号为 0 到 $N - 1$ 。客户 i 有 W_i 元存款，且客户之间的存款金额互不相同。

小 P 是小 Y 的深度合作伙伴，他希望知道哪个客户的存款最多。小 P 无法直接获取客户的存款金额，但他可以依次发送若干次**请求**，每次请求包含若干个**查询**，每个查询是一个二元组 (i, j) ，表示小 P 想知道客户 i 和客户 j 的存款金额哪个更多。如果 $W_i > W_j$ ，小 Y 会回答 i ，否则回答 j 。

小 P 的**请求数** t 和所有请求的**查询**次数总和 s 有上限，他希望你帮他写一个程序，来找到存款最多的客户。

实现细节

请确保提交的程序开头包含 `#include "richest.h"`。

选手不需要，也不应该实现主函数。

选手需要实现以下函数：

```
1 | int richest(int N, int T, int S);
```

- N 表示客户的数量；
- T 表示对于当前函数调用，**请求数** t 不应超过此值；
- S 表示对于当前函数调用，所有请求的**查询**次数总和 s 不应超过此值；
- 该函数需要返回存款最多的客户的编号；
- 对于每个测试点，该函数会被交互库调用恰好 10 次。

选手可以通过调用以下函数向交互库发送一次**请求**：

```
1 | std::vector<int> ask(std::vector<int> a, std::vector<int> b);
```

- 在调用 `ask` 函数时需要保证传入参数 a 和 b 的长度相同，且其中的每个元素都必须是小于 N 的非负整数，表示该**请求**中的所有**查询**；
- 该函数会返回一个长度与 a 和 b 相同的 `std::vector<int> c`，其中 $c[i]$ 表示在客户 $a[i]$ 和 $b[i]$ 中存款金额较多的那一位的编号。

题目保证在规定的**请求与查询**次数限制下，交互库运行所需的时间不超过 10 秒；交互库使用的内存大小固定，且不超过 256 MiB。

测试程序方式

试题目录下的 `grader.cpp` 是提供的交互库参考实现，最终测试时所用的交互库实现与该参考实现有所不同，因此选手的解法不应该依赖交互库实现。

选手可以在本题目目录下使用如下命令编译得到可执行程序：

```
1 | g++ grader.cpp richest.cpp -o richest -O2 -std=c++14 -static
```

对于编译得到的可执行程序：

- 可执行文件将从**标准输入**读入以下格式的数据：
 - 输入的第一行包含四个非负整数 N, T, S, R ，其中 R 是交互库生成测试数据的随机种子。
- 输入完成后，交互库将调用 10 次函数 `richest`，用输入的参数生成的测试数据进行测试。
`richest` 函数返回后，交互库会输出以下信息：
 - 输出的前 10 行中，每行首先包含三个整数 r, t, s ，表示该次执行的结果，其中 r 是 `richest` 函数的返回值， t 和 s 的含义如题目描述中所示，然后包含该次运行的正确性等信息。
 - 输出的第 11 行包含 10 次运行的总信息。

交互示例

假设可执行文件生成的测试数据为 $N = 4, W = [101, 103, 102, 100], T = 100, S = 100$ 。

下面是一个正确的交互过程：

选手程序	交互库	说明
调用 <code>richest(4, 100, 100)</code>	开始测试	
调用 <code>ask([0, 2], [1, 3])</code>	返回 $[1, 2]$	$W_0 < W_1, W_2 > W_3$
调用 <code>ask([0, 2, 3], [1, 1, 1])</code>	返回 $[1, 1, 1]$	$W_0 < W_1, W_2 < W_1, W_3 < W_1$
运行结束并返回 1	向屏幕打印交互结果	交互结束，结果正确

在这个例子中， $r = 1, t = 2, s = 5$ ，满足请求与查询次数的限制。

下发文件说明

在本试题目录下：

1. `grader.cpp` 是提供的交互库参考实现。
2. `richest.h` 是头文件，选手不用关心具体内容。
3. `template_richest.cpp` 是提供的示例代码，选手可在此代码的基础上实现。

选手注意对所有下发文件做好备份。最终评测时只测试本试题目录下的 `richest.cpp`，对该程序以外文件的修改不会影响评测结果。

数据范围

本题共 2 个测试点，每个测试点的分值和数据范围见下表。

测试点编号	分值	$N =$	$T =$	$S =$
1	15	1 000	1	499 500
2	85	1 000 000	20	2 000 000

评分方式

注意：

- 选手不应当通过非法方式获取交互库的内部信息，如试图直接读取数组 W 的值，或直接与标准输入、输出流进行交互。此类行为将被视为作弊；
- 最终的评测交互库与样例交互库的实现不同，且可能是适应性的：在不与 `ask` 此前返回的结果相矛盾的前提下，它可能会动态调整 W 的值。

本题首先会受到和传统题相同的限制，例如编译错误会导致整道题目得 0 分，运行时错误、超过时间限制、超过空间限制等会导致相应测试点得 0 分等。选手只能在程序中访问自己定义的和交互库给出的变量及其对应的内存空间，尝试访问其他空间将可能导致编译错误或运行错误。

在每次 `richest` 函数调用中，选手程序使用的请求次数 t 和所有请求的查询次数总和 s 需在对应限制下，否则将会获得 0 分。

在上述条件基础上：

- 在测试点 1 中，程序得到满分当且仅当 `ask` 函数调用均合法且 `richest` 函数返回的答案正确；
- 在测试点 2 中，程序得到的分数将按照以下方式计算：
 - 若 `ask` 函数调用不合法，则获得 0 分；
 - 若 `ask` 函数调用均合法，设 $\max t$ 表示多次调用 `richest` 函数时所得的 t 的最大值， $\max s$ 表示 s 的最大值，则程序将获得 $\lfloor 85 \cdot f(\max t) \cdot g(\max s) \rfloor$ 分，其中 f 与 g 的计算方式如下表所示：

$\max t$	$f(\max t)$
$\max t \leq 8$	1
$9 \leq \max t \leq 20$	$1 - \frac{1}{4}\sqrt{\max t - 8}$
$\max s$	$g(\max s)$
$\max s \leq 1\,099\,944$	1
$1\,099\,945 \leq \max s \leq 1\,100\,043$	$1 - \frac{1}{6}\log_{10}(\max s - 1\,099\,943)$
$1\,100\,044 \leq \max s \leq 2\,000\,000$	$\frac{2}{3} - \frac{1}{1\,500}\sqrt{\max s - 1\,100\,043}$

以下是测试点 2 中，不同的 t 和 s 对得分影响的示例：

$\max t$	$\max s$	测试点 2 的得分
= 20	$\leq 1\,099\,944$	11
= 19		14
= 18		17
= 17		21
= 16		24
= 15		28
= 14		32
= 13		37
= 12		42
= 11		48
= 10		54
= 9		63
≤ 8	$\in [1\,099\,974, 1\,099\,978]$	63
≤ 8	$\in [1\,099\,969, 1\,099\,973]$	64
≤ 8	$\in [1\,099\,965, 1\,099\,968]$	65
≤ 8	$\in [1\,099\,962, 1\,099\,964]$	66

$\max t$	$\max s$	测试点 2 的得分
≤ 8	$\in [1\,099\,959, 1\,099\,961]$	67
≤ 8	$\in [1\,099\,957, 1\,099\,958]$	68
≤ 8	$\in [1\,099\,955, 1\,099\,956]$	69
≤ 8	$\in [1\,099\,953, 1\,099\,954]$	70
≤ 8	$= 1\,099\,952$	71
≤ 8	$= 1\,099\,951$	72
≤ 8	$\in [1\,099\,949, 1\,099\,950]$	73
≤ 8	$= 1\,099\,948$	75
≤ 8	$= 1\,099\,947$	76
≤ 8	$= 1\,099\,946$	78
≤ 8	$= 1\,099\,945$	80
≤ 8	$\leq 1\,099\,944$	85

Message

Aisha and Basma are two friends who correspond with each other. Aisha has a message M , which is a sequence of S bits (i.e., zeroes or ones), that she would like to send to Basma. Aisha communicates with Basma by sending her **packets**. A packet is a sequence of 31 bits indexed from 0 to 30. Aisha would like to send the message M to Basma by sending her some number of packets.

Unfortunately, Cleopatra compromised the communication between Aisha and Basma and is able to **taint** the packets. That is, in each packet Cleopatra can modify bits on exactly 15 indices. Specifically, there is an array C of length 31, in which every element is either 0 or 1, with the following meaning:

- $C[i] = 1$ indicates that the bit with index i can be changed by Cleopatra. We call these indices **controlled** by Cleopatra.
- $C[i] = 0$ indicates that bit with index i cannot be changed by Cleopatra.

The array C contains precisely 15 ones and 16 zeroes. While sending the message M , the set of indices controlled by Cleopatra stays the same for all packets. Aisha knows precisely which 15 indices are controlled by Cleopatra. Basma only knows that 15 indices are controlled by Cleopatra, but she does not know which indices.

Let A be a packet that Aisha decides to send (which we call the **original packet**). Let B be the packet that is received by Basma (which we call the **tainted packet**). For each i , such that $0 \leq i < 31$:

- if Cleopatra does not control the bit with index i ($C[i] = 0$), Basma receives bit i as sent by Aisha ($B[i] = A[i]$),
- otherwise, if Cleopatra controls the bit with index i ($C[i] = 1$), the value of $B[i]$ is decided by Cleopatra.

Immediately after sending each packet, Aisha learns what the corresponding tainted packet is.

After Aisha sends all the packets, Basma receives all the tainted packets **in the order they were sent** and has to reconstruct the original message M .

Your task is to devise and implement a strategy that would allow Aisha to send the message M to Basma, so that Basma can recover M from the tainted packets. Specifically, you should implement two procedures. The first procedure performs the actions of Aisha. It is given a message M and

the array C , and should send some packets to transfer the message to Basma. The second procedure performs the actions of Basma. It is given the tainted packets and should recover the original message M .

Implementation Details

The first procedure you should implement is:

```
void send_message(std::vector<bool> M, std::vector<bool> C)
```

- M : an array of length S describing the message that Aisha wants to send to Basma.
- C : an array of length 31 indicating the indices of bits controlled by Cleopatra.
- This procedure may be called **at most 2100 times** in each test case.

This procedure should call the following procedure to send a packet:

```
std::vector<bool> send_packet(std::vector<bool> A)
```

- A : an original packet (an array of length 31) representing the bits sent by Aisha.
- This procedure returns a tainted packet B representing the bits that will be received by Basma.
- This procedure can be called at most 100 times in each invocation of `send_message`.

The second procedure you should implement is:

```
std::vector<bool> receive_message(std::vector<std::vector<bool>> R)
```

- R : an array describing the tainted packets. The packets originate from packets sent by Aisha in one `send_message` call and are given **in the order they were sent** by Aisha. Each element of R is an array of length 31, representing a tainted packet.
- This procedure should return an array of S bits that is equal to the original message M .
- This procedure may be called **multiple times** in each test case, **exactly once** for each corresponding `send_message` call. The **order of receive_message procedure calls** is not necessarily the same as the order of the corresponding `send_message` calls.

Note that in the grading system the `send_message` and `receive_message` procedures are called in **two separate programs**.

Constraints

- $1 \leq S \leq 1024$
- C has exactly 31 elements, out of which 16 are equal to 0 and 15 are equal to 1.

Subtasks and Scoring

If in any of the test cases, the calls to the procedure `send_packet` do not conform to the rules mentioned above, or the return value of any of the calls to procedure `receive_message` is incorrect, the score of your solution for that test case will be 0.

Otherwise, let Q be the maximum number of calls to the procedure `send_packet` among all invocations of `send_message` over all test cases. Also let X be equal to:

- 1, if $Q \leq 66$
- 0.95^{Q-66} , if $66 < Q \leq 100$

Then, the score is calculated as follows:

Subtask	Score	Additional Constraints
1	$10 \cdot X$	$S \leq 64$
2	$90 \cdot X$	No additional constraints.

Note that in some cases the behaviour of the grader can be **adaptive**. This means that the values returned by `send_packet` may depend not just on its input arguments but also on many other things, including the inputs and return values of the prior calls to this procedure and pseudo-random numbers generated by the grader. The grader is **deterministic** in the sense that if you run it twice and in both runs you send the same packets, it will make the same changes to them.

Example

Consider the following call.

```
send_message([0, 1, 1, 0],  
             [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
              1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

The message that Aisha tries to send to Basma is $[0, 1, 1, 0]$. The bits with indices from 0 to 15 cannot be changed by Cleopatra, while the bits with indices from 16 to 30 can be changed by Cleopatra.

For the sake of this example, let us assume that Cleopatra fills consecutive bits she controls with alternating 0 and 1, i.e. she assigns 0 to the first index she controls (index 16 in our case), 1 to the second index she controls (index 17), 0 to the third index she controls (index 18), and so on.

Aisha can decide to send two bits from the original message in one packet as follows: she will send the first bit at the first 8 indices she controls and the second bit at the following 8 indices she

controls.

Aisha then chooses to send the following packet:

```
send_packet([0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Note that Cleopatra can change bits with the last 15 indices, so Aisha can set them arbitrarily, as they might be overwritten. With the assumed strategy of Cleopatra, the procedure returns: $[0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]$.

Aisha decides to send the last two bits of M in the second packet in a similar way as before:

```
send_packet([1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
             0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

With the assumed strategy of Cleopatra, the procedure returns: $[1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]$.

Aisha can send more packets, but she chooses not to.

The grader then makes the following procedure call:

```
receive_message([[0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1,
                  0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
                 [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0,
                  0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]])
```

Basma recovers message M as follows. From each packet she takes the first bit that occurs twice in a row, and the last bit that occurs twice in a row. That is, from the first packet, she takes bits $[0, 1]$, and from the second packet she takes bits $[1, 0]$. By putting them together, she recovers the message $[0, 1, 1, 0]$, which is the correct return value for this call to `receive_message`.

It can be shown that with the assumed strategy of Cleopatra and for messages of length 4, this approach of Basma correctly recovers M , regardless of the value of C . However, it is not correct in the general case.

Sample Grader

The sample grader is not adaptive. Instead, Cleopatra fills consecutive bits she controls with alternating 0 and 1 bits, as described in the example above.

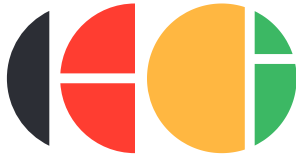
Input format: **The first line of the input contains an integer T , specifying the number of scenarios.** T scenarios follow. Each of them is provided in the following format:

```
S
M[0] M[1] ... M[S-1]
C[0] C[1] ... C[30]
```

Output format: The sample grader writes the result of each of the T scenarios in the same order as they are provided in the input in the following format:

```
K L
D[0] D[1] ... D[L-1]
```

Here, K is the number of calls to `send_packet`, D is the message returned by `receive_message` and L is its length.



How to Avoid Disqualification in 75 Easy Steps (avoid)

You are standing in front of the open safe, a medal in your hand. But your triumph turns to despair as you look around: a message on a tie you found in the room reveals that your assistant exposed your plan to the Scientific Committee! Now the two committee chairmen are hiding in the building to stop you from escaping...

Luckily, you have R vacuum cleaner robots left over from your trade with your fellow contestants. You want to use these robots to locate the two chairmen so that you can avoid them during your escape. You can instruct each robot to scout several of 1 000 positions where the chairmen could be located. Unfortunately, the software of these robots is quite simple.* Each robot can only detect whether or not *there is at least one chairman at its scouted positions*.

To make things worse, every robot needs a full hour to scout its positions before returning with its result to you. Because this will drain the robot's battery, *you can send out each robot only once*.

As you don't want to be late for the evening activities, you want to know the positions of the chairmen *after at most H hours*. In particular, you might be forced to send out several robots at once without waiting for the previous robots to return. You can assume that the two chairmen stay at the same positions all the time.†

Write a program which plans this scouting mission and determines where the two chairmen are located.

Communication

This is a communication task. You must implement the function `pair<int, int> scout(int R, int H)` where R and H are as described above. For each testcase, this function is called exactly once and should return a pair of two integers $1 \leq a, b \leq 1\,000$ ($a = b$ is allowed), the positions of the two chairmen. Inside `scout`, you can use the following other functions provided by the grader:

- ▶ `void send(vector<int> P)` sends a robot to scout the positions $P[0], \dots, P[k - 1]$ (where k is the length of the array P). The positions $P[i]$ must be pairwise distinct integers between 1 and 1 000. You can call this function at most R times per testcase.
- ▶ `vector<int> wait()` waits an hour. This function returns an array with exactly one entry for each robot sent out one hour ago (by a call to `send` after the previous call to `wait` or after the beginning of the program). The entry at index i is 1 if the $(i + 1)$ -th of these robots has detected at least one chairman at its scouted positions, and 0 otherwise. You can call this function at most H times per testcase.

If any of your function calls does not satisfy the above constraints, your program will be immediately terminated and judged as **Not correct** for the respective testcase. You must not write to standard output or read from standard input; otherwise, you may receive the verdict **Security violation!**. You are however free to write to the standard error stream (`stderr`).

You must include the file `avoid.h` in your source code. To test your program locally, you can link it with `sample_grader.cpp`, which can be found in the attachment for this task in cms. See below for a description of the sample grader, and see `sample_grader.cpp` for instructions on how to run it with your program. The attachment also contains a sample implementation as `avoid_sample.cpp`.

* After all, you sold all the fancy ones to the other contestants!

† Due to the late-night task preparations they are simply too exhausted to move.

Constraints and grading

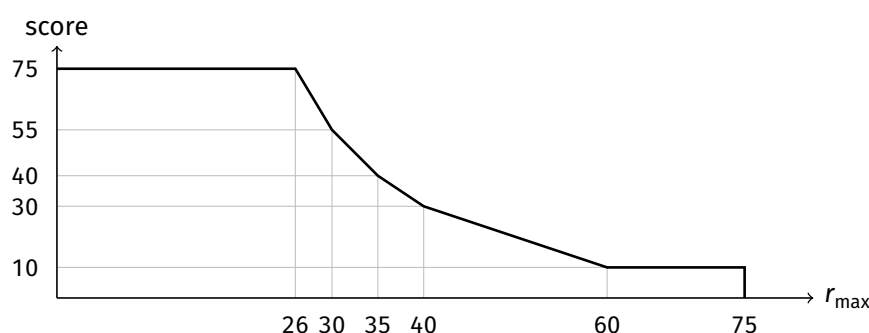
Subtask 1 (10 points). $R = 10, H = 1$ and both chairmen are located at the same position.

Subtask 2 (5 points). $R = H = 20$

Subtask 3 (10 points). $R = 30, H = 2$

Subtask 4 (75 points). $R = 75, H = 1$

Partial scoring. In Subtask 4, your actual score depends on the maximum number $r_{\max}$ of robots sent out over all testcases in this subtask according to the following piecewise linear function:



In particular, to get full score you must not make more than 26 calls to *send* per testcase in the last subtask. You can also find a table listing all the individual scores in the attachments for this task as `score_table.txt`.

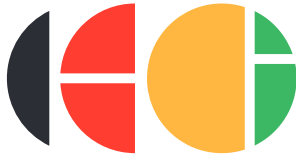
Example interaction

Consider a testcase with $R = 75$ and $H = 20$ where the chairmen are located at positions 13 and 37. First, the grader calls your function *scout* as *scout*(75, 20). Then, an interaction between your program and the grader could look as follows:

Your program	Return value	Explanation
<i>send</i> ({42, 13, 37})	—	send a robot to positions 13, 37 and 42
<i>send</i> ({47, 11})	—	send a robot to positions 11 and 47
<i>wait</i> ()	{1, 0}	wait an hour for the return of the robots; only the first robot has detected a chairman
<i>send</i> ({42})	—	send a robot to position 42
<i>wait</i> ()	{0}	wait an hour; there is no chairman at position 42
return {13, 37}	—	you are convinced that the chairmen are located at positions 13 and 37 the solution is correct and is accepted

Returning the pair {37, 13} would be accepted as well. Note that the above queries are of course not sufficient to determine the positions of the chairmen with certainty: For example, both being at position 37 or one chairman being at position 13 while the other is at position 100 would also be consistent with all answers to *wait*, so the grader could also have rejected this solution.

The above interaction is reproduced by `avoid_sample.cpp` on the public testcase.



CEOI 2023

Central-European Olympiad in Informatics
Magdeburg | Germany | August 13 - 19

Day 2
Task: **avoid**
Language: **en**

Grader

The sample grader first expects on standard input the integers R and H and the positions a and b of the chairmen ($1 \leq a, b \leq 1\,000$). Then, the grader calls $\text{scout}(R, H)$ and writes to standard output a protocol of all grader functions called by your program. Upon termination, it writes one of the following messages to standard output:

Invalid input. The input to the grader via standard input was not of the above format.

Invalid send. You called *send* with invalid parameters.

Out of robots. You called *send* more than R times.

Out of time. You called *wait* more than H times.

Wrong answer. The positions returned by *scout* are not the positions of the chairmen.

Correct: r robot(s) used, h hour(s) passed. The positions returned by *scout* are the positions of the chairmen, there were r calls to *send*, and there were h calls to *wait*.

In contrast, the grader actually used to judge your program will only output **Not correct** (for any of the above errors), **Security violation!**, or **Correct: r robot(s) used, h hour(s) passed**. Moreover, the grader is *adaptive*, i.e. the positions of the chairmen may depend on the behavior of your program in the current as well as in earlier runs. Both the sample grader and the grader used to judge your program will terminate your program automatically whenever one of the above errors occurs.

Limits

Time: 0.25 s

Memory: 512 MiB

Problem L. 采矿

题目背景

在精心地规划完工人的移动路线，执行完所有计划之后，你终于有钱了。你承包下了一个更大的矿坑，并购买了更先进的设备。

但是开始运行了你才发现，一部分运输矿物的通道居然装反了！还好它们本来就是可以反向的，并且中控的系统可以让你轻易地操作。

然而，现在最大的问题是，你刚刚接手这个矿坑，你甚至连它长什么样都不知道，也就更不知道每个开关是对应哪一条运输通道的。

时间就是金钱，你想要尽快摸清整个矿坑的结构以及所有开关与通道的对应关系。

题目描述

这是一道交互题。

已知你的矿坑有 n 个节点，编号为 $1 \sim n$ 。它们通过 $n - 1$ 条运输通道连成一个树形结构。

运输通道都是单向的。对于一条从节点 u 到节点 v 的运输通道，可以将所有由节点 u 产出的矿或运送到节点 u 的矿以较快的速度运送到节点 v 。如果一个节点有多条以其为起点的运输通道，那么会把这些矿平均分配给这些运输通道。

中控的系统包含 $n - 1$ 个开关和一个监视器。开关的编号为 $1 \sim n - 1$ ，每个开关可以拨到 0 或 1 的位置。 $n - 1$ 个开关和 $n - 1$ 条运输通道一一对应，但你并不知道它们的对应关系。你只知道，假设编号为 i 的开关对应的运输通道在被装上去时是从 u_i 到 v_i 的，那么当开关拨到 0 的时候，它的运输方向和它被装上去时相同；当开关拨到 1 的时候，它的运输方向会变成从 v_i 到 u_i 。你的监视器可以监控到达每个节点的矿分别来自多少个不同的节点，也就是说，有多少个节点（包括其本身）能够通过运输通道把矿运输到这个节点。

当你调整完开关的位置后，需要等一段时间，监视器的结果才会趋于稳定，这时你的读数才是有意义的。所以为了避免浪费太多时间，你希望在 50 次读数之内确定你想知道的所有信息。

输入格式

从标准输入读入数据。

开始时你需要输入一个正整数 n 。保证 $2 \leq n \leq 10000$ 。

之后的输入会基于你的输出生成读数。你的读数结果是一行 n 个正整数，其中第 i 个表示到达节点 i 的矿来自多少个不同的节点。

每个测试点中，矿坑的连接方式和所有运输通道刚装上去时的方向都是固定的，也就是说，这些不会因为你的输出而动态修改为另外一种符合之前所有回答的方案。

输出格式

输出到标准输出。

当你需要调整开关并等待读数时，输出一行 `? s`，其中 `s` 为一个长为 $n - 1$ 的 `01` 串，其中第 i 位表示编号为 i 的开关拨到的位置。然后交互库会在你的标准输入中给出监视器趋于稳定之后的读数结果。你最多只能读数 50 次。

当你已经知道了所有通道的信息时，输出一行 `! u1 v1 ... un-1 vn-1`，其中 u_i, v_i 表示编号为 i 的开关对应的运输通道被装上去时的方向是从节点 u_i 到节点 v_i 的。

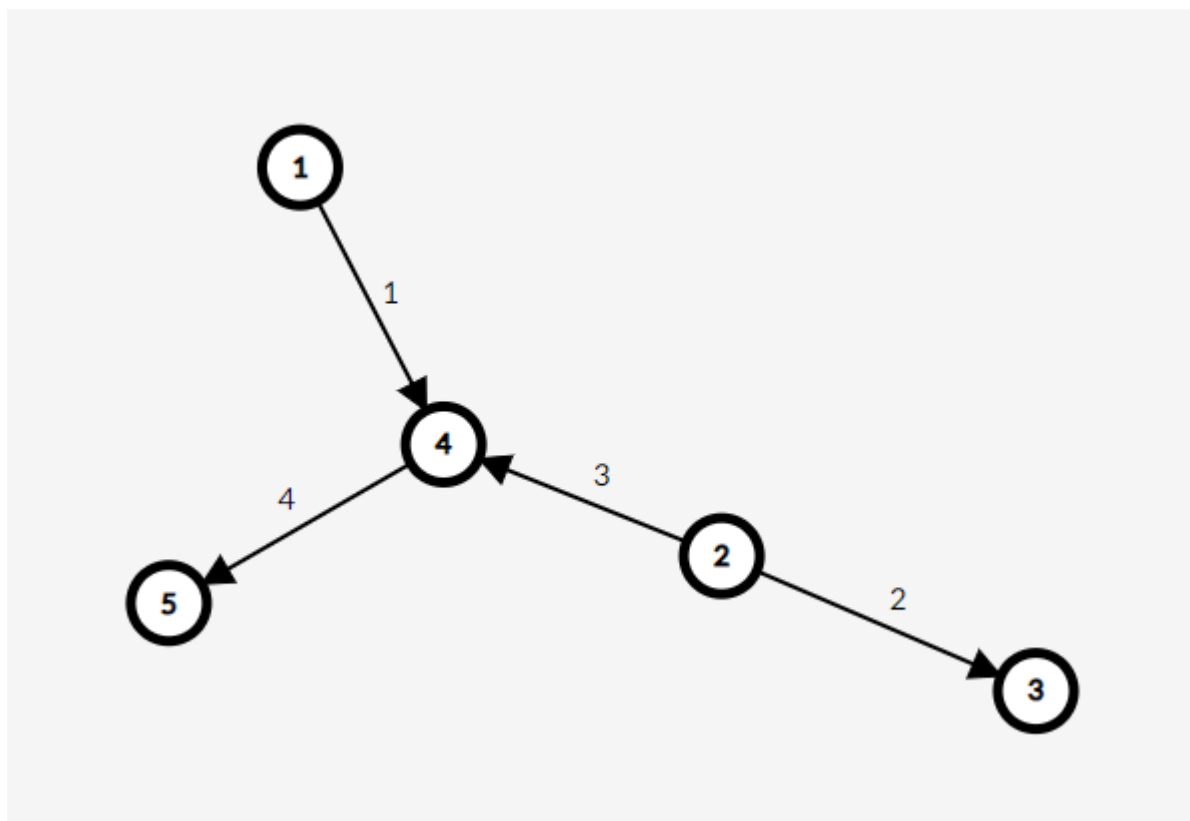
在输出一行之后，你需要刷新输出缓冲区，否则评测结果可能会变成 `TLE`。刷新输出缓冲区的方式为：

- C : `fflush(stdout);`
- C++ : `fflush(stdout);` 或 `std::cout.flush();` 或使用 `std::endl` 换行
- Java : `System.out.flush();`
- Python : `sys.stdout.flush()`

样例

Input:	Output:
5	
	? 0110
1 4 1 2 3	
	? 0000
1 1 2 3 4	
	! 1 4 2 3 2 4 4 5

解释



通道的初始方向如上图所示。通道上的数字代表和通道对应的开关的编号。

样例只是用来说明输入输出格式和读数结果，并不意味着这次读数能够推出答案。

评分方式

交互库的运行时间和内存不计入时间和内存限制。

若超出读数次数限制、最后的回答错误或输出格式错误，评测结果均为 **WA**。